

Technical Specification & Methodological Reference:

**Exact Augmentations, Full End-to-End Pipelines, Architectures,
Learning Rates, and Hyperparameters Across Adaptation Paradigms 1–4**

Mohammad Yavari

September 2026

Repository: <https://github.com/myavarih/CLIP-LoRA>

1. Executive Overview & Paradigm Taxonomy	2
1.1. Parameter Footprint and Trainable Parameter Counts	2
2. Data Preprocessing & Augmentation Pipelines	4
2.1. Exact Mathematical Specifications of Augmentation Transforms	4
2.2. Pipeline Implementation Listings	6
3. End-to-End Pipeline Specifications by Paradigm	8
3.1. Run 1: Infix CoOp + Residual Class Tokens + Ordinal Loss	8
3.1.1 Architecture and Forward Computation	8
3.1.2 Optimization and Parameter Grouping	8
3.2. Run 2: Plain Dual LoRA Baseline	10
3.2.1 Architecture and Forward Computation	10
3.2.2 Optimization Specification	10
3.3. Run 3: CoOp-CSC Dual LoRA	11
3.3.1 Architecture and Forward Computation	11
3.3.2 Optimization and Staging Mechanics	11
3.4. Run 4: Plain LoRA + Residual Class Tokens (ResCls)	13
3.4.1 Architecture and Forward Computation	13
3.4.2 Optimization Specification	13
4. Comprehensive Hyperparameter Reference Table	15
5. Command Line Execution Reference	15
5.1. Run 1: Infix CoOp + Ordinal (Vision LoRA)	15
5.2. Run 2: Plain Dual LoRA Baseline	16
5.3. Run 3: CoOp-CSC Dual LoRA	16
5.4. Run 4: Plain LoRA + Residual Class Tokens (ResCls)	17

1. Executive Overview & Paradigm Taxonomy

This technical document details the implementation mechanics, data transformation pipelines, model architectures, loss formulations, optimization configurations, and exact hyperparameter settings across the four fine-tuning paradigms evaluated in the benchmark suite (Runs 1–4).

Each run represents a distinct balance between visual adaptation, language adaptation, continuous prompt tuning, and class token conditioning on top of the frozen OpenAI CLIP ViT-B/16 foundation model. Table 1 specifies the core algorithmic components of each paradigm.

TABLE 1. Systematic architectural and algorithmic matrix across Evaluated Runs 1–4.

Run	Paradigm	Vision Adapter	Text Adapter	Prompt & Class Formulation	Loss Objective
Run 1	Infix CoOp + Ordinal	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	Frozen Backbone	Infix continuous prompt ($M=4$) + Learnable residual class tokens Δe_k	$\mathcal{L}_{ce} + \lambda_{ord}\mathcal{L}_{ord}$ ($\lambda_{ord}=1.0$ or 2.0)
Run 2	Plain Dual LoRA	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	Handcrafted natural template, frozen BPE token embeddings	Standard $\mathcal{L}_{ce}$
Run 3	CoOp-CSC Dual LoRA	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	Class-Specific Context ($K \times M \times d$ continuous prompt vectors)	Standard $\mathcal{L}_{ce}$
Run 4	Plain LoRA + ResCls	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	LoRA (W_q, W_v , $r=2, \alpha=1.0$)	Handcrafted natural template + Learnable residual class tokens δ_k	Standard $\mathcal{L}_{ce}$

Note: ViT-B/16 operates with vision dimension $d_v = 768$ (projected to 512) and text dimension $d_t = 512$. All LoRA modules inject into attention projection matrices with rank $r = 2$ and scaling factor $\alpha = 1.0$.

1.1. Parameter Footprint and Trainable Parameter Counts

Table 2 quantifies the exact parameter allocations across each paradigm, illustrating how the parameter count varies as a function of the number of classes $K \in \{5, 6, 196\}$.

TABLE 2. Trainable parameter breakdown across adaptation paradigms for Walnut ($K = 6$), Piarom Date ($K = 5$), and Stanford Cars ($K = 196$).

Parameter Component	Run 1	Run 2	Run 3	Run 4
Vision LoRA ($W_q, W_v \times 12$ layers)	147,456	147,456	147,456	147,456
Text LoRA ($W_q, W_v \times 12$ layers)	0 (Frozen)	98,304	98,304	98,304
Continuous Context Vectors	2,048 ($M = 4$)	0	$K \times 2,048$	0
Class Token Residual Offsets	$K \times 512$	0	0	$K \times 512$
Total Trainable ($K = 5$, Piarom)	152,064	245,760	255,992	248,320
Total Trainable ($K = 6$, Walnut)	152,576	245,760	258,048	248,832
Total Trainable ($K = 196$, Cars)	249,856	245,760	647,168	346,112

Note: In Run 3, class-specific context parameters scale directly with class cardinality K

($K \times M \times d = 196 \times 4 \times 512 = 401,408$ parameters for Stanford Cars), whereas Run 2 maintains a strictly constant parameter count irrespective of K .

2. Data Preprocessing & Augmentation Pipelines

The benchmark evaluations compare two distinct image preprocessing and data augmentation regimes:

1. **Initial Augmentation Suite (Seeds 11, 12, 13):** Configured via `-resize_mode crop`. This utilizes standard random resized cropping during training and center cropping during testing.
2. **Fixed Augmentation Suite (Seeds 21, 22, 23):** Configured via `-resize_mode pad`. This introduces aspect-ratio-preserving letterbox square padding (SquarePad), random vertical flipping, sub-pixel translation shifts, additive Gaussian noise, and impulse salt-and-pepper noise.

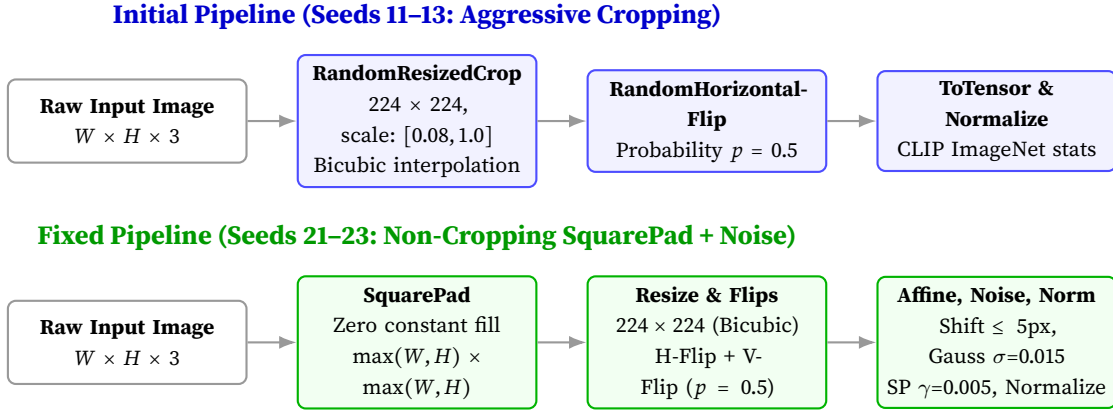


FIGURE 1. Architectural dataflow diagram comparing the Initial cropping augmentation pipeline against the Fixed aspect-ratio-preserving letterbox and noise augmentation pipeline.

2.1. Exact Mathematical Specifications of Augmentation Transforms

Table 3 provides the precise parameter configurations, operational bounds, and mathematical formulations for each transform in the codebase.

TABLE 3. Detailed technical specification of individual transformation steps in the pipelines.

Transform Name	Applied Suite	Parameters	Mathematical Definition / Operation
SquarePad	Fixed (Train/Test)	fill = 0, constant	Pads image tensor/PIL to $\max(W, H) \times \max(W, H)$ with symmetric margins: $h_p = \lfloor (\max - W)/2 \rfloor$, $v_p = \lfloor (\max - H)/2 \rfloor$.
RandomResizedCrop	Initial (Train)	size = 224, scale = (0.08, 1.0)	Random area sample $\alpha \sim \mathcal{U}(0.08, 1.0)$, aspect ratio $r \sim \mathcal{U}(3/4, 4/3)$, cropped and scaled to 224×224 .
CenterCrop	Initial (Test)	size = 224	Central rectangular window crop: $\left[\frac{W-224}{2}, \frac{H-224}{2}, \frac{W+224}{2}, \frac{H+224}{2} \right]$ after resizing shortest edge to 224.
Resize	Fixed (Train/Test)	size = (224, 224)	Bicubic interpolation resizing square padded canvas to model input tensor grid.
RandomHorizontalFlip	Both (Train)	$p = 0.5$	Left-right reflection matrix applied with probability 0.5.
RandomVerticalFlip	Fixed (Train)	$p = 0.5$	Top-bottom reflection applied with probability 0.5 (enabled on agricultural datasets; disabled on Cars).
RandomAffine	Fixed (Train)	degrees = 0, trans = (5/224, 5/224)	Translation displacement $(\Delta x, \Delta y) \in [-5, +5]$ pixels with bicubic interpolation and zero boundary fill.
AddGaussianNoise	Fixed (Train)	$\mu = 0.0, \sigma = 0.015, p = 0.5$	$x' = \text{clip}(x + \epsilon, 0, 1)$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ applied on tensor prior to normalization.
AddSaltAndPepperNoise	Fixed (Train)	$\gamma = 0.005, \text{ratio} = 0.5, p = 0.5$	Pixel mask $M \sim \mathcal{U}(0, 1)^{H \times W}$; sets pixels to 1.0 if $M < 0.0025$ and 0.0 if $0.0025 \leq M < 0.005$.
Normalize	Both (Train/Test)	$\mu = (0.481, 0.458, 0.408)$ $\sigma = (0.269, 0.261, 0.276)$	Standard OpenAI CLIP channel-wise normalization: $x'_c = (x_c - \mu_c) / \sigma_c$.

2.2. Pipeline Implementation Listings

The complete transform compositions implemented in `my_impl/clip/clip.py` and `my_impl/main.py` are defined as follows:

Listing 1. Implementation of SquarePad, Noise transforms, and the full preprocessing pipelines in `clip.py` and `main.py`.

```
1 class SquarePad(object):
2     """Pads image to square canvas preserving aspect ratio without cropping."""
3     def __init__(self, fill=0, padding_mode='constant'):
4         self.fill = fill
5         self.padding_mode = padding_mode
6
7     def __call__(self, image):
8         w, h = image.size if hasattr(image, 'size') else (image.shape[-1], image.
9             shape[-2])
10        max_wh = max(w, h)
11        hp = (max_wh - w) // 2
12        vp = (max_wh - h) // 2
13        padding = (hp, vp, max_wh - w - hp, max_wh - h - vp)
14        return F.pad(image, padding, fill=self.fill, padding_mode=self.padding_mode)
15
16 # Fixed Augmentation Pipeline (Seeds 21, 22, 23)
17 train_transform_fixed = transforms.Compose([
18     transforms.Lambda(lambda img: img.convert('RGB')),
19     SquarePad(),
20     transforms.Resize((224, 224), interpolation=transforms.InterpolationMode.BICUBIC
21         ),
22     transforms.RandomHorizontalFlip(p=0.5),
23     transforms.RandomVerticalFlip(p=0.5), # Disabled for Stanford Cars
24     transforms.RandomAffine(degrees=0, translate=(5/224.0, 5/224.0),
25         interpolation=transforms.InterpolationMode.BICUBIC, fill
26         =0),
27     transforms.ToTensor(),
28     AddGaussianNoise(mean=0.0, std=0.015, p=0.5),
29     AddSaltAndPepperNoise(amount=0.005, salt_ratio=0.5, p=0.5),
30     transforms.Normalize(mean=(0.48145466, 0.4578275, 0.40821073),
31         std=(0.26862954, 0.26130258, 0.27577711))
32 ])
33
34 # Fixed Evaluation Transform (Test & Val Splits)
35 test_transform_fixed = transforms.Compose([
36     transforms.Lambda(lambda img: img.convert('RGB')),
37     SquarePad(),
38     transforms.Resize((224, 224), interpolation=transforms.InterpolationMode.BICUBIC
39         ),
40     transforms.ToTensor(),
41     transforms.Normalize(mean=(0.48145466, 0.4578275, 0.40821073),
```

38

```
std=(0.26862954, 0.26130258, 0.27577711))
```

39])

3. End-to-End Pipeline Specifications by Paradigm

3.1. Run 1: Infix CoOp + Residual Class Tokens + Ordinal Loss

3.1.1. Architecture and Forward Computation

Run 1 combines continuous prefix/suffix prompt tuning, residual class token learning, vision encoder LoRA adaptation, and a partial ordinal cost penalty:

1. **Vision Encoder:** LoRA low-rank decomposition matrices $A_v \in \mathbf{R}^{r \times d_{\text{in}}}$ and $B_v \in \mathbf{R}^{d_{\text{out}} \times r}$ are attached to the query (W_q) and value (W_v) projections across all 12 self-attention blocks of CLIP’s ViT-B/16. The modified linear projection computes:

$$(1) \quad h_v = W_0 x + \frac{\alpha}{r} B_v A_v x, \quad \text{with } r = 2, \alpha = 1.0$$

2. **Text Encoder:** The Transformer backbone is kept strictly frozen.
3. **Infix PromptLearner:** An infix template $\mathcal{T} = \text{"photo_of_a_}\{\}_\text{<item>"}$ is parsed into prefix and suffix tokens. $M = 4$ continuous context vectors $\mathbf{V} = [\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, \mathbf{v}_4]$ are initialized from the tokenized prefix and suffix embeddings.
4. **Residual Class Tokens:** For each class $k \in \{1, \dots, K\}$, the base BPE embedding vector $e_k^0 \in \mathbf{R}^d$ is conditioned with a dedicated learnable residual vector $\Delta e_k \in \mathbf{R}^d$:

$$(2) \quad e_k = e_k^0 + \Delta e_k, \quad \Delta e_k \text{ initialized to } \mathbf{0}$$

The full sequence of token embeddings fed to the text encoder for class k is:

$$(3) \quad \mathbf{P}_k = [\mathbf{e}_{\text{SOS}}, \mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3, e_k, \mathbf{v}_4, \mathbf{e}_{\text{EOS}}, \mathbf{e}_{\text{PAD}}, \dots] \in \mathbf{R}^{77 \times 512}$$

5. **Loss Function:** The model optimizes a joint objective combining standard cross-entropy and expected ordinal risk using a domain cost matrix $\mathbf{C} \in \mathbf{R}^{K \times K}$:

$$(4) \quad \mathcal{L} = \mathcal{L}_{\text{ce}} + \lambda_{\text{ord}} \mathcal{L}_{\text{ord}} = -\log P(y^* | \mathbf{x}) + \lambda_{\text{ord}} \sum_{j=1}^K P(y = j | \mathbf{x}) C_{y^*, j}$$

3.1.2. Optimization and Parameter Grouping

- Vision LoRA parameters (147,456 params): $\eta_{\text{loa}} = 2 \times 10^{-4}$, weight decay = 10^{-2} .
- Continuous Context vectors (2,048 params): $\eta_{\text{ctx}} = 2 \times 10^{-4}$, weight decay = 0.0.
- Residual class parameters ($K \times 512$ params): $\eta_{\text{class}} = 1 \times 10^{-3}$, weight decay = 0.0.
- Scheduler: CosineAnnealingLR to $\eta_{\text{min}} = 10^{-6}$ over $250 \times k$ iterations.

Listing 2. Run 1 Parameter groups and Composite Criterion initialization in train_coop.py.

```
1 trainable_params = [  
2     {'params': get_lora_parameters(clip_model), 'lr': 2e-4, 'weight_decay': 1e-2},  
3     {'params': [prompt_learner.ctx], 'lr': 2e-4, 'weight_decay': 0.0},  
4     {'params': prompt_learner.class_deltas.parameters(), 'lr': 1e-3, 'weight_decay':  
        0.0}  
5 ]  
6 optimizer = torch.optim.AdamW(trainable_params, betas=(0.9, 0.999), lr=2e-4)  
7 criterion = CompositeCriterion(  
8     base_loss_type='ce', use_ordinal=True,  
9     cost_matrix=cost_matrix, lambda_ord=1.0 # (or 2.0)  
10 )
```

3.2. Run 2: Plain Dual LoRA Baseline

3.2.1. Architecture and Forward Computation

Run 2 constitutes the standard parameter-efficient fine-tuning baseline, adapting both visual and linguistic representations symmetrically without any continuous prompt vectors:

1. **Vision Encoder:** LoRA modules ($r = 2$, $\alpha = 1.0$, dropout $p = 0.25$) injected into W_q, W_v across all 12 visual Transformer blocks.
2. **Text Encoder:** LoRA modules ($r = 2$, $\alpha = 1.0$, dropout $p = 0.25$) injected into W_q, W_v across all 12 textual Transformer blocks (98,304 parameters).
3. **Prompt Architecture:** Handcrafted static prompt templates (e.g., "a photo of a {} walnut, a type of walnut."). Class names are tokenized with frozen standard CLIP BPE tokenization.
4. **Loss Function:** Standard cross-entropy classification:

$$(5) \quad \mathcal{L} = \mathcal{L}_{\text{ce}} = -\log \frac{\exp(100 \cdot \cos(f_{\mathbf{x}}, w_{y^*}))}{\sum_{k=1}^K \exp(100 \cdot \cos(f_{\mathbf{x}}, w_k))}$$

3.2.2. Optimization Specification

- Parameter group: All LoRA parameters (245,760 total parameters across both encoders).
- Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, weight decay = 10^{-2}).
- Learning rate: $\eta = 2 \times 10^{-4}$ with CosineAnnealingLR decay down to 10^{-6} .
- Batch size: 32, total iterations = $250 \times k$.

Listing 3. Run 2 Optimizer initialization in lora.py.

```
1 list_lora_layers = apply_lora(args, clip_model) # Injects both vision & text
2 mark_only_lora_as_trainable(clip_model)
3 optimizer = torch.optim.AdamW(
4     get_lora_parameters(clip_model),
5     lr=2e-4, weight_decay=1e-2, betas=(0.9, 0.999)
6 )
7 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, total_iters,
8     eta_min=1e-6)
```

3.3. Run 3: CoOp-CSC Dual LoRA

3.3.1. Architecture and Forward Computation

Run 3 couples Dual LoRA adaptation with Class-Specific Context (CSC) continuous prompt vectors:

1. **Dual LoRA Adaptation:** LoRA ($r = 2, \alpha = 1.0$) injected into W_q, W_v of both the Vision Transformer and Text Transformer (245,760 adapter parameters).
2. **Class-Specific Context (CSC):** Rather than sharing M continuous context vectors across all classes, every class k is allocated an independent set of $M = 4$ continuous context vectors:

$$(6) \quad \mathbf{V}^{(k)} = [\mathbf{v}_{k,1}, \mathbf{v}_{k,2}, \mathbf{v}_{k,3}, \mathbf{v}_{k,4}] \in \mathbf{R}^{4 \times 512}$$

The total parameter count of the context matrix is $K \times M \times d$.

3. **Prompt Assembly:** For each class k , the token embedding tensor fed to the text encoder is constructed as:

$$(7) \quad \mathbf{P}_k = [\mathbf{e}_{\text{SOS}}, \mathbf{v}_{k,1}, \mathbf{v}_{k,2}, \mathbf{v}_{k,3}, \mathbf{v}_{k,4}, \mathbf{e}_{\text{cls}}^{(k)}, \mathbf{e}_{\text{EOS}}, \dots] \in \mathbf{R}^{77 \times 512}$$

where $\mathbf{e}_{\text{cls}}^{(k)}$ is the frozen BPE embedding of the k -th class name.

4. **Loss Function:** Standard cross-entropy loss $\mathcal{L}_{\text{ce}}$ (ordinal loss flag was omitted during execution).

3.3.2. Optimization and Staging Mechanics

- Trainable parameters: LoRA parameters (245,760) with $\eta = 2 \times 10^{-4}$ and weight decay 10^{-2} ; CSC context parameters ($K \times 2,048$) with $\eta = 2 \times 10^{-4}$ and weight decay 0.0.
- Staged training option: The codebase supports freezing CSC prompts after N_{csc} iterations via `-csc_iters`. In standard benchmark runs, ‘csc_iters’ defaults to $N_{\text{iters}} = 250$, training end-to-end throughout all iterations.

Listing 4. Run 3 PromptLearner initialization with Class-Specific Context (CSC) in `train_coop.py`.

```
1 prompt_learner = PromptLearner(  
2     clip_model=clip_model,  
3     classnames=dataset.classnames,  
4     n_ctx=4,  
5     csc=True,                                # Enables K x M x d independent context vectors  
6     learn_class_tokens=False  
7 ).cuda()  
8  
9 trainable_params = [  
10     {'params': get_lora_parameters(clip_model), 'lr': 2e-4, 'weight_decay': 1e-2},  
11     {'params': [prompt_learner.ctx], 'lr': 2e-4, 'weight_decay': 0.0}  
12 ]
```

```
13 optimizer = torch.optim.AdamW(trainable_params, betas=(0.9, 0.999), lr=2e-4)
```

3.4. Run 4: Plain LoRA + Residual Class Tokens (ResCls)

3.4.1. Architecture and Forward Computation

Run 4 investigates the hypothesis that preserving the full natural language sentence template while providing localized capacity on the class name token embedding avoids prompt distortion:

1. **Dual LoRA Adaptation:** Identical to Run 2, attaching LoRA ($r = 2, \alpha = 1.0$) to W_q, W_v in both vision and text encoders (245,760 parameters).
2. **Residual Class Tokens:** Utilizes `ResidualTemplatePromptLearner` initialized with the complete natural template sentence:

$$(8) \quad \mathcal{T} = \text{"a photo of a \{ \} <item>, a type of <item>."}$$

The template tokens remain fixed, but for each class k , an additive residual vector $\delta_k \in \mathbf{R}^{L_k \times d}$ is learned over the class token span:

$$(9) \quad \mathbf{e}_{\text{seq}}^{(k)}[s_k : s_k + L_k] = \mathbf{e}_{\text{base}}^{(k)}[s_k : s_k + L_k] + \delta_k$$

where s_k is the exact BPE token starting index of class k , and L_k is the token length of the class name.

3. **Loss Function:** Standard cross-entropy classification $\mathcal{L}_{\text{ce}}$.

3.4.2. Optimization Specification

- LoRA parameter group: $\eta_{\text{lora}} = 2 \times 10^{-4}$, weight decay = 10^{-2} .
- Residual class parameters ($K \times 512$ params): $\eta_{\text{class}} = 1 \times 10^{-3}$ (5× higher learning rate), weight decay = 0.0.
- Optimizer: AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$).
- Scheduler: CosineAnnealingLR over $250 \times k$ iterations.

Listing 5. Run 4 `ResidualTemplatePromptLearner` and parameter groups in `train_coop.py`.

```
1 prompt_learner = ResidualTemplatePromptLearner(  
2     clip_model=clip_model,  
3     classnames=dataset.classnames,  
4     template=dataset.template[0],  
5     learn_template_tokens=False,      # Static natural template tokens  
6     learn_class_tokens=True          # Additive class residual offsets  
7 ).cuda()  
8  
9 trainable_params = [  
10     {'params': get_lora_parameters(clip_model),      'lr': 2e-4, 'weight_decay':  
11         1e-2},
```

```
11     {'params': prompt_learner.class_deltas.parameters(), 'lr': 1e-3, 'weight_decay':  
12         0.0}  
13 ]  
14 optimizer = torch.optim.AdamW(trainable_params, betas=(0.9, 0.999), lr=2e-4)  
15 scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(  
16     optimizer,  
17     T_max=total_iters,  
18     eta_min=1e-6  
19 )
```

4. Comprehensive Hyperparameter Reference Table

Table 4 provides the definitive mapping of all hyperparameters across the four experimental runs and training pipelines.

TABLE 4. Comprehensive hyperparameter reference across all four adaptation paradigms.

Hyperparameter	Run 1	Run 2	Run 3	Run 4	CLI Flag / Source
CLIP Backbone	ViT-B/16	ViT-B/16	ViT-B/16	ViT-B/16	--backbone
LoRA Target Encoders	Vision only	Vision + Text	Vision + Text	Vision + Text	--encoder
LoRA Rank (r)	2	2	2	2	--r
LoRA Scaling (α)	1.0	1.0	1.0	1.0	--alpha
LoRA Projections	W_q, W_v	W_q, W_v	W_q, W_v	W_q, W_v	--paramsqv
LoRA Dropout	0.25	0.25	0.25	0.25	--dropout_rate
LoRA Learning Rate	2×10^{-4}	2×10^{-4}	2×10^{-4}	2×10^{-4}	--lr
LoRA Weight Decay	10^{-2}	10^{-2}	10^{-2}	10^{-2}	Fixed in code
Context Tokens (M)	4	0 (N/A)	4	0 (N/A)	--n_ctx
Context Sharing	Unified (Infix)	N/A	Class-Specific	N/A	--csc
Context Learning Rate	2×10^{-4}	N/A	2×10^{-4}	N/A	--lr
Context Weight Decay	0.0	N/A	0.0	N/A	Fixed in code
Class Token Learning	Residual (Δe_k)	None (Frozen)	None (Frozen)	Residual (δ_k)	--learn_class_tokens
Class LR (η_{class})	1×10^{-3}	N/A	N/A	1×10^{-3}	--lr_class
Class Weight Decay	0.0	N/A	N/A	0.0	Fixed in code
Loss Formulation	$\mathcal{L}_{\text{ce}} + \lambda_{\text{ord}} \mathcal{L}_{\text{ord}}$	$\mathcal{L}_{\text{ce}}$	$\mathcal{L}_{\text{ce}}$	$\mathcal{L}_{\text{ce}}$	--use_ordinal
Ordinal Weight (λ_{ord})	1.0 (or 2.0)	N/A	N/A	N/A	--lambda_ord
Cost Matrix Metric	Expected Cost	N/A	N/A	N/A	--ordinal_loss_type
Logit Scale (τ^{-1})	100.0	100.0	100.0	100.0	Fixed CLIP scale
Batch Size (B)	32	32	32	32	--batch_size
Iterations / Shot	250	250	250	250	--n_iters
Total Iterations (N)	$250 \times k$	$250 \times k$	$250 \times k$	$250 \times k$	Calculated
Optimizer	AdamW	AdamW	AdamW	AdamW	torch.optim.AdamW
Betas (β_1, β_2)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	Fixed in code
LR Scheduler	Cosine Anneal	Cosine Anneal	Cosine Anneal	Cosine Anneal	CosineAnnealingLR
Minimum LR (η_{min})	10^{-6}	10^{-6}	10^{-6}	10^{-6}	Fixed in code
Precision	FP16 (AMP)	FP16 (AMP)	FP16 (AMP)	FP16 (AMP)	torch.amp.autocast

5. Command Line Execution Reference

Below are the exact execution commands invoking `my_impl/run_experiments.py` for each paradigm across the benchmark suite:

5.1. Run 1: Infix CoOp + Ordinal (Vision LoRA)

```
1 python3 my_impl/run_experiments.py \
2   --root_path /kaggle/input/datasets/emmwhy3/few-shot-data/FewShotData \
```

```

3  --dataset walnut \
4  --shots 1 2 4 8 16 32 \
5  --seed 21 22 23 \
6  --method coop_lora \
7  --ctx_init "photo_of_a_{}_walnut" \
8  --learn_class_tokens \
9  --use_ordinal \
10 --lambda_ord 1.0 \
11 --encoder vision \
12 --r 2 --alpha 1.0 \
13 --lr 2e-4 --lr_class 1e-3 \
14 --n_iters 250 --batch_size 32 \
15 --output_dir /kaggle/working/experiments_output/run1_infix_coop_ordinal_walnut \
16 --checkpoints_dir /kaggle/working/checkpoints/run1_infix_coop_ordinal_walnut

```

5.2. Run 2: Plain Dual LoRA Baseline

```

1 python3 my_impl/run_experiments.py \
2  --root_path /kaggle/input/datasets/emmwhy3/few-shot-data/FewShotData \
3  --dataset walnut \
4  --shots 1 2 4 8 16 32 \
5  --seed 21 22 23 \
6  --method lora \
7  --encoder both \
8  --r 2 --alpha 1.0 \
9  --lr 2e-4 \
10 --n_iters 250 --batch_size 32 \
11 --output_dir /kaggle/working/experiments_output/run2_plain_lora_walnut \
12 --checkpoints_dir /kaggle/working/checkpoints/run2_plain_lora_walnut

```

5.3. Run 3: CoOp-CSC Dual LoRA

```

1 python3 my_impl/run_experiments.py \
2  --root_path /kaggle/input/datasets/emmwhy3/few-shot-data/FewShotData \
3  --dataset walnut \
4  --shots 1 2 4 8 16 32 \
5  --seed 21 22 23 \
6  --method csc_lora \
7  --csc --n_ctx 4 \
8  --encoder both \
9  --r 2 --alpha 1.0 \
10 --lr 2e-4 \
11 --n_iters 250 --batch_size 32 \
12 --output_dir /kaggle/working/experiments_output/run3_coop_csc_dual_lora_walnut \
13 --checkpoints_dir /kaggle/working/checkpoints/run3_coop_csc_dual_lora_walnut

```

5.4. Run 4: Plain LoRA + Residual Class Tokens (ResCls)

```
1 python3 my_impl/run_experiments.py \  
2     --root_path /kaggle/input/datasets/emmwhy3/few-shot-data/FewShotData \  
3     --dataset walnut \  
4     --shots 1 2 4 8 16 32 \  
5     --seed 21 22 23 \  
6     --method res_cls_lora \  
7     --encoder both \  
8     --learn_class_tokens \  
9     --r 2 --alpha 1.0 \  
10    --lr 2e-4 --lr_class 1e-3 \  
11    --n_iters 250 --batch_size 32 \  
12    --output_dir /kaggle/working/experiments_output/run4_plain_lora_rescls_walnut \  
13    --checkpoints_dir /kaggle/working/checkpoints/run4_plain_lora_rescls_walnut
```